

WSL + VSCode + tmux + Claude Code로 구축하는 윈도우 AI 개발 환경 완전 가이드

이 글에서 얻을 수 있는 것: WSL + VSCode Remote + tmux + Claude Code를 조합한 개발 환경 구축 방법과, tmux를 활용해 Claude Code에서 여러 터미널을 제어하는 오케스트레이션 기법을 배울 수 있습니다.

2026/2/8 업데이트: 이 글에서 구축하는 WSL + tmux 환경은 Claude Code의 Agent Teams split panes 모드와 직결됩니다. tmux 환경이 갖춰지면 윈도우에서도 Agent Teams 에이전트들이 실시간으로 대화하고 협력하는 모습을 하나의 화면에서 여러 분할 창으로 확인할 수 있습니다.

자세한 내용은 「Agent Teams의 split panes 모드」 섹션을 참고하세요.

보너스 팁: WSL에서 Claude Code를 사용하다 보면, 스크린샷을 붙여넣을 수 없어서 불편하지 않으신가요? "이 에러 화면 좀 봐줘"가 안 됩니다. 이 문제를 해결하는 도구를 만들었습니다 → [screenshot-to-wsl](#)

왜 WSL을 사용하는가

윈도우에서 개발할 때 WSL(Windows Subsystem for Linux)을 사용하는 이유는 다음과 같습니다.

리눅스 툴체인을 그대로 사용 가능

- Node.js, Python, Docker 등이 네이티브로 동작
- 셸 스크립트나 CLI 도구를 macOS/Linux와 동일하게 사용 가능
- 팀 내 macOS/Linux 개발자와 환경을 통일하기 쉬움

성능이 좋음

- WSL 내부 파일 시스템(~/)은 Windows 측(/mnt/c/)보다 빠름
- `npm install`이나 `git` 작업이 체감상 수 배 빨라짐

Claude Code와의 궁합이 좋음

- Claude Code는 Linux/macOS 환경을 전제로 설계되어 있음
- bash 커맨드나 tmux 조작이 그대로 동작

왜 tmux를 사용하는가

터미널 멀티플렉서로 tmux를 사용하는 이유는 여러이지만, 특히 Claude Code와의 조합에서 진가를 발휘합니다.

일반적인 장점

- SSH 연결이 끊기거나 터미널을 종료해도 세션이 유지됨
- tmux-resurrect로 PC 재시작 후에도 세션 복원 가능
- 여러 창(pane)에서 작업을 병렬로 진행하면서 키보드만으로 전환 가능

Claude Code와의 조합으로 최강이 됨

tmux의 `send-keys` 커맨드로 다른 창(pane)에 커맨드를 전송할 수 있습니다. 이것이 Claude Code와 결합하면 강력해집니다.

여러 창에 Claude Code를 실행해 각각 다른 태스크를 지시할 수 있습니다:

```
# pane 1의 Claude Code에 코드 리뷰 요청
tmux send-keys -t .1 "claude '이 PR을 리뷰해줘'" Enter

# pane 2의 Claude Code에 테스트 작성 요청
tmux send-keys -t .2 "claude '이 함수의 유닛 테스트를 작성해줘'" Enter
```

즉, 여러 Claude Code를 병렬로 실행하여 한쪽에는 코드 리뷰, 다른 쪽에는 테스트 작성 같은 분업이 가능해집니다.

`Ctrl+b s`로 세션 목록을 표시해 전환하면서 진행 상황을 모니터링하면, 여러 AI 에이전트를 동시에 실행하는 「오케스트레이션」이 가능합니다.

셋업 편

처음 한 번만 수행하는 설정입니다.

1. WSL 확인 및 셋업

설치 여부 확인

```
# PowerShell에서
wsl -l -v
```

Ubuntu가 표시되면 OK. 없으면 설치:

```
wsl --install -d Ubuntu
```

WSL 메모리 제한 (권장)

WSL2는 방치하면 메모리를 계속 점유하므로 제한을 걸어 둡니다.

`%USERPROFILE%\wslconfig` 파일을 생성합니다:

```
[wsl2]
memory=16GB
swap=4GB
```

설정 후 WSL을 재시작:

```
wsl --shutdown
```

2. 기본 도구 설치

WSL 터미널(Ubuntu)에서 실행:

```
# 패키지 업데이트
sudo apt update && sudo apt upgrade -y

# 기본 도구
sudo apt install -y git curl wget build-essential

# tmux
sudo apt install -y tmux

# Node.js (nvm 경유)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash
source ~/.bashrc
nvm install --lts

# Python (필요한 경우)
sudo apt install -y python3 python3-pip python3-venv
```

3. tmux 설정

TPM (Tmux Plugin Manager) 설치

```
git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm
```

tmux 설정 파일

~/.tmux.conf 를 생성합니다:

```
cat > ~/.tmux.conf << 'EOF'
# Plugins
set -g @plugin 'tmux-plugins/tpm'
set -g @plugin 'tmux-plugins/tmux-sensible'
set -g @plugin 'tmux-plugins/tmux-resurrect' # session save/restore

# Mouse
set -g mouse on

# Pane split
bind | split-window -h -c "#{pane_current_path}"
bind - split-window -v -c "#{pane_current_path}"

# Pane navigation (vim style)
bind h select-pane -L
bind j select-pane -D
bind k select-pane -U
```

```
bind l select-pane -R

# Colors
set -g default-terminal "screen-256color"
set -g status-bg colour235
set -g status-fg white

# Active pane highlight
set -g pane-active-border-style fg=green
set -g window-style 'bg=colour235'
set -g window-active-style 'bg=black'

# TPM init (keep at bottom)
run '~/.tmux/plugins/tpm/tpm'
EOF
```

플러그인 설치

```
# tmux를 시작
tmux

# 플러그인 설치
# Ctrl+b I (대문자 I) 를 누름
```

TPM 키바인딩

조작	키
플러그인 설치	Ctrl+b I (대문자 I)
플러그인 업데이트	Ctrl+b U
플러그인 삭제	Ctrl+b Alt+u

tmux-resurrect (세션 저장·복원)

조작	키
저장	Ctrl+b Ctrl+s
복원	Ctrl+b Ctrl+r

pane 레이아웃, 각 pane의 디렉터리, 실행 중인 커맨드도 복원됩니다. PC 재시작 후에도 복원 가능합니다.

4. Docker 셋업

방법 A: Docker Desktop (간편, 개인 사용자용)

1. Windows 측에 Docker Desktop 설치

2. Settings → Resources → WSL Integration → Ubuntu 활성화

3. WSL 내에서 확인:

```
docker --version
docker run hello-world
```

방법 B: WSL 내에 직접 설치 (무료, 기업용)

```
# 설치
sudo apt install -y docker.io docker-compose

# 사용자를 docker 그룹에 추가
sudo usermod -aG docker $USER

# 서비스 시작
sudo service docker start

# WSL 재시작 후 확인
docker run hello-world
```

자동 시작이 필요한 경우 `~/.bashrc`에 추가:

```
# Docker auto start
if service docker status 2>&1 | grep -q "is not running"; then
    sudo service docker start
fi
```

[보충 설명 - 추가된 내용] Docker Desktop은 대규모 기업에서는 유료 라이선스가 필요합니다(직원 수 250명 이상 또는 연매출 1천만 달러 이상). 사내 정책에 따라 방법 B나 Podman 등의 대안을 검토하세요.

5. VSCode Remote WSL 셋업

Windows 측

1. [VSCode](#) 설치
2. 확장 기능 'WSL' 설치 (Microsoft 공식)

연결 확인

```
# WSL 내에서 프로젝트 폴더로 이동
cd ~/projects

# VSCode 열기
code .
```

VSCode가 실행되고 좌측 하단에 'WSL: Ubuntu'가 표시되면 OK.

6. Claude Code 셋업

설치 (네이티브 인스톨러 권장)

```
# WSL 내에서 실행
curl -fsSL https://claude.ai/install.sh | bash

# 셸 설정 재로드
source ~/.bashrc
```

확인

```
claude --version

# 문제가 있으면 진단
claude doctor
```

기존 npm 버전에서 마이그레이션

npm 버전을 사용하고 있었다면:

```
claude install
```

이것으로 네이티브 버전으로 마이그레이션됩니다. 설정 파일은 유지됩니다.

7. 셸 설정 (별칭 등)

~/.bashrc 에 추가:

```
cat >> ~/.bashrc << 'EOF'

# tmux
alias t='tmux'
alias ta='tmux attach -t'
alias tl='tmux ls'
alias tn='tmux new -s'
alias tk='tmux kill-session'

# tmux send-keys shortcut
ts() {
  tmux send-keys -t . $1 "${@:2}" Enter
}

# projects
alias proj='cd ~/projects'

# dev session
alias dev='~/bin/dev-session.sh'
```

```
EOF
```

```
source ~/.bashrc
```

ts 함수 사용법:

```
ts 0 "claude"      # pane 0에 커맨드 전송
ts 1 "npm run dev" # pane 1에 커맨드 전송
ts 2 "npm test"    # pane 2에 커맨드 전송
```

tk 로 세션 종료:

```
tk      # 현재 세션 종료
tk dev  # dev 세션 종료
```

개발용 tmux 세션 스크립트 (선택 사항)

매번 tmux에서 pane을 분할하는 게 번거로운 분을 위한 스크립트입니다.

`dev ~/projects/my-app` 이라고 입력하기만 하면 3-pane 레이아웃이 즉시 생성됩니다:

```
+-----+-----+
|           | 0.1   |
|           | (작업용) |
| 0.0       +-----+
| (작업용)   | 0.2   |
|           | (작업용) |
+-----+-----+
```

사용 예시:

- `0.0`: Claude Code 실행 (넓게 사용)
- `0.1`: `npm run dev` 등 서버 시작
- `0.2`: git 조작, 테스트 실행 등

자동으로 뭔가를 실행하는 게 아니라, 셸 3개가 나란히 열릴 뿐입니다.

불필요하면 건너뛰어도 OK. 수동으로 pane을 분할해도 동일하게 사용할 수 있습니다.

```
mkdir -p ~/bin

cat > ~/bin/dev-session.sh << 'EOF'
#!/bin/bash

SESSION="dev"
PROJECT_DIR="${1:-$HOME/projects}"

# Kill existing session
tmux kill-session -t $SESSION 2>/dev/null
```

```
# Create new session
tmux new-session -d -s $SESSION -c $PROJECT_DIR

# Rename window
tmux rename-window -t $SESSION:0 'main'

# Split panes
# Right pane
tmux split-window -h -t $SESSION:0 -c $PROJECT_DIR
# Split right pane vertically
tmux split-window -v -t $SESSION:0.1 -c $PROJECT_DIR

# Layout:
# +-----+-----+
# |           | 0.1 |
# |           | (작업용) |
# | 0.0       +-----+
# | (작업용)  | 0.2 |
# |           | (작업용) |
# +-----+-----+

# Attach
tmux select-pane -t $SESSION:0.0
tmux attach -t $SESSION
EOF

chmod +x ~/bin/dev-session.sh
```

사용 편

매일 참고하는 섹션입니다.

8. WSL 시작하기

매일 작업은 여기서 시작합니다.

방법 1: Windows Terminal에서

1. Windows Terminal 열기
2. 탭의 「v」에서 「Ubuntu」 선택

방법 2: 시작 메뉴에서

「Ubuntu」를 검색해서 실행

방법 3: PowerShell에서

```
wsl
```

시작하면 홈 디렉터리(~)에 있을 것입니다.

9. 프로젝트 배치

주의: 프로젝트는 반드시 WSL 내(`~/`)에 배치할 것. `/mnt/c/`는 느립니다.

```
# 프로젝트용 디렉터리 생성
mkdir -p ~/projects
cd ~/projects

# 리포지토리 클론
git clone https://github.com/your/project.git
cd project

# VSCode로 열기
code .
```

Windows 측에서 접근하고 싶은 경우

탐색기 주소창에 입력:

```
\\wsl$\\Ubuntu\\home\\사용자이름\\projects
```

또는 WSL 내에서:

```
explorer.exe .
```

10. tmux 기본 조작

시작과 종료

```
# 시작
tmux

# 이름 붙인 세션으로 시작
tmux new -s dev

# 세션 목록
tmux ls

# 세션에 연결
tmux attach -t dev

# 디태치 (세션을 유지한 채 빠져나오기)
Ctrl+b d

# 현재 pane 종료
exit

# 세션째 종료 (모든 pane 종료)
```

```
tmux kill-session

# 외부에서 특정 세션 종료
tmux kill-session -t dev

# 모든 세션 종료
tmux kill-server
```

Pane 조작

조작	키
세로 분할	Ctrl+b % (또는 Ctrl+b)
가로 분할	Ctrl+b " (또는 Ctrl+b -)
Pane 이동	Ctrl+b 화살표키 (또는 Ctrl+b h/j/k/l)
Pane 삭제	Ctrl+b x
Pane 최대화/원래대로	Ctrl+b z
Pane 번호 표시	Ctrl+b q

윈도우 조작

조작	키
새 윈도우	Ctrl+b c
다음 윈도우	Ctrl+b n
이전 윈도우	Ctrl+b p
윈도우 목록	Ctrl+b w

세션 전환

조작	키
세션 목록	Ctrl+b s

11. tmux send-keys로 pane 간 통신

Pane 지정 방법

세션명:윈도우번호.pane번호

예: `dev:0.1` = dev 세션의 0번 윈도우의 1번 pane

같은 세션 내라면 단축 표기 가능

```
# 세션명과 윈도우 번호를 생략하고, pane 번호만
tmux send-keys -t .1 "echo hello" Enter
tmux send-keys -t .2 "npm run dev" Enter

# 별칭 설정이 되어 있다면 (셋업 편 참고)
ts 1 "npm run dev"
ts 2 "npm test"
```

기본적인 사용법

```
# 다른 pane에 커맨드 보내기
tmux send-keys -t dev:0.1 "echo hello" Enter

# 서버 시작
tmux send-keys -t dev:0.1 "npm run dev" Enter

# 테스트 실행
tmux send-keys -t dev:0.2 "npm run test:watch" Enter

# 여러 커맨드
tmux send-keys -t dev:0.1 "cd ~/projects/my-app && npm install" Enter

# 프로세스 중지 (Ctrl+C 전송)
tmux send-keys -t dev:0.1 "C-c"
```

Pane 번호 확인

```
# tmux 내에서
Ctrl+b q
```

12. Claude Code + tmux로 오케스트레이션

Claude Code 입력 팁

여러 줄로 지시를 입력할 때, `Shift+Enter`로 줄 바꿈이 안 되는 경우(전송되어 버리는 경우)에는 `Ctrl+J`로 줄 바꿈할 수 있습니다.

Claude Code에 "다른 pane을 조작해줘"라고 요청하기

Claude Code는 bash 커맨드를 실행할 수 있습니다. 즉, `tmux send-keys`도 실행할 수 있습니다.

이를 이용해 Claude Code에 다음과 같이 지시할 수 있습니다:

```
pane 0.1에서 서버를 재시작하고, pane 0.2에서 테스트를 실행해줘.
tmux send-keys를 사용해줘.
```

그러면 Claude Code가 다음과 같은 커맨드를 실행해 줍니다:

```
# pane 0.1의 서버를 재시작
tmux send-keys -t .1 "C-c" # Ctrl+C로 정지
tmux send-keys -t .1 "npm run dev" Enter

# pane 0.2에서 테스트 실행
tmux send-keys -t .2 "npm test" Enter
```

즉, 본인은 Claude Code와 대화하는 것만으로, 다른 pane 조작을 Claude Code에게 맡길 수 있습니다.

전형적인 개발 플로우

```
# 1. 개발 세션 시작
dev ~/projects/my-app

# 2. 레이아웃
# +-----+-----+
# |           | 0.1   |
# |           | (작업용) |
# | 0.0       +-----+
# | (작업용)   | 0.2   |
# |           | (작업용) |
# +-----+-----+

# 3. pane 0.0에서 Claude Code 시작
claude

# 4. Claude Code가 다른 pane을 조작
#   - 0.1: 서버 시작/재시작
#   - 0.2: 테스트 실행, git 조작 등
```

13. Agent Teams의 split panes 모드

Claude Code의 실험적 기능인 Agent Teams는 리드 에이전트가 여러 팀원을 실행해 병렬로 작업을 진행하는 구조입니다. Agent Teams에는 split panes 모드가 있어, tmux를 사용해 각 팀원을 별도 pane에 표시할 수 있습니다.

즉, WSL + tmux 환경을 구축해 두면 윈도우에서도 Agent Teams 에이전트들이 실시간으로 대화하고 협력하는 모습을 하나의 화면에서 확인할 수 있습니다.

설정 방법

Claude Code의 `settings.json`에 다음을 추가합니다:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1",
    "CLAUDE_CODE_SPAWN_BACKEND": "tmux"
  }
}
```

`CLAUDE_CODE_SPAWN_BACKEND`를 `tmux`로 설정하면, 팀원들이 각각 별도의 `tmux` pane에서 실행됩니다.

실제 동작

Agent Teams를 시작하면 리드 에이전트가 태스크를 분할하고, 팀원을 자동으로 별도 pane에 실행합니다.

`Ctrl+b` 화살표키 (또는 `Ctrl+b h/j/k/l`)로 pane을 전환하면서 각 에이전트의 작업을 실시간으로 모니터링할 수 있습니다.

일반적인 서브 에이전트에서는 결과만 부모에게 반환되지만, Agent Teams에서는 팀원 간에 직접 메시지로 연결하기 때문에 그 협력 프로세스 전체를 눈앞에서 확인할 수 있는 것이 큰 차이점입니다.

Agent Teams의 자세한 내용(서브 에이전트와의 차이, Delegate 모드 등)은 「Claude Code 태스크 관리·Agent Teams 시대에 SDD 도구는 필요한가」를 참고하세요.

트러블슈팅

클립보드의 이미지를 Claude Code에 붙여넣을 수 없음

WSL 위의 Claude Code는 Windows 클립보드에서 직접 이미지를 받을 수 없습니다.

이 문제를 해결하는 도구를 만들었습니다:

동작 방식:

1. `Ctrl+Shift+v`를 누르면 클립보드의 이미지가 파일로 저장됨
2. WSL 호환 경로(`/mnt/c/...` 형식)가 클립보드에 복사됨
3. 그 경로를 Claude Code에 붙여넣으면 이미지를 읽어들이 수 있음

필요한 것: AutoHotkey v2.0

WSL이 느림

프로젝트가 `/mnt/c/`에 있지 않은지 확인하세요. `~/` 하위로 이동하면 해결됩니다.

Docker가 동작하지 않음

```
# 서비스 상태 확인
sudo service docker status

# 시작
sudo service docker start
```

tmux에서 복사가 잘 안 됨

마우스 모드가 활성화되어 있는 경우, `Shift`를 누르면서 선택하면 Windows 측 클립보드를 사용할 수 있습니다.

tmux 설정 파일에서 오류 발생

Windows에서 편집하면 줄 바꿈 코드가 이상해지는 경우가 있습니다(CRLF 문제):

```
# 수정
sed -i 's/\r$//' ~/.tmux.conf

# 또는
sudo apt install dos2unix
dos2unix ~/.tmux.conf
```

VSCode Remote에서 확장 기능이 동작하지 않음

WSL 측에 확장 기능을 재설치해야 하는 경우가 있습니다.

tmux가 중첩됨

이미 tmux 세션 내에 있는 상태에서 `tmux` 를 실행하면 경고가 표시됩니다:

```
# 확인
echo $TMUX

# 기존 세션에서 빠져나가기
Ctrl+b d

# 세션 목록
tmux ls

# 기존 세션에 연결
tmux attach -t 세션명
```

마치며

솔직히 말하면, 최고의 개발 경험을 원한다면 Mac을 구입하는 것이 가장 좋을 수도 있습니다.

macOS라면 Claude Code CLI뿐만 아니라 브라우저의 Claude(claude.ai)나 Claude for Desktop, 나아가 MCP(Model Context Protocol)를 사용한 각종 도구 연동까지 모든 것이 매끄럽게 통합되어 있습니다. 터미널도 네이티브 Unix 환경이기 때문에 WSL과 같은 가상화 레이어를 거칠 필요도 없습니다.

그렇지만 Windows 사용자가 당장 Mac으로 전환하는 것은 현실적으로 어려운 경우도 많습니다.

이 글에서 소개한 WSL + VSCode Remote + tmux + Claude Code 조합이라면, Windows에서도 충분히 쾌적한 개발 경험을 실현할 수 있습니다. 특히 tmux를 사용한 다중 Claude Code 오케스트레이션은 macOS에서도 동일하게 사용할 수 있는 기법입니다.

먼저 이 환경을 시도해 보고, AI와 함께 코드를 작성하는 새로운 개발 스타일을 경험해 보세요.

[보충 설명 - 추가된 내용] 한국 개발 환경에서 자주 쓰이는 도구 연동 팁:

- **GitHub Copilot과 병행 사용:** VSCode에 GitHub Copilot 확장이 설치되어 있어도, Claude Code는 터미널에서 독립적으로 동작하므로 충돌 없이 함께 사용할 수 있습니다.
- **사내 VPN 환경:** 일부 기업 VPN에서는 WSL2의 네트워크가 제한될 수 있습니다. 이 경우 `wsl -shutdown` 후 VPN 연결 상태에서 WSL을 재시작해 보세요.
- **Windows 11 권장:** WSL2의 GUI 앱 지원(WSLg)과 `systemd` 지원이 Windows 11에서 더 안정적입니다.

MIT by @gaebalai